

Base de datos y conexión con Supabase

Resumen de implementación, migraciones, verificaciones y pendientes

Fecha: 1 de octubre de 2026 | Motor: PostgreSQL administrado por Supabase

ESTADO FUNCIONAL: LISTO PARA PRUEBAS LOCALES

La conexión a Supabase fue comprobada, se adaptó el esquema anterior con migraciones EF Core, se conservaron los registros existentes y se verificaron el inicio de sesión y las consultas del panel. La compilación y las pruebas automatizadas también pasan.

NO ES AÚN UNA CONFIGURACIÓN SEGURA DE PRODUCCIÓN

La contraseña administrativa actual es débil y la credencial de base de datos se manejó durante la configuración local. Deben rotarse las credenciales expuestas y configurar secretos y migraciones de forma segura antes de publicar.

1. Qué encontramos

El backend ya utilizaba Entity Framework Core 8 y Npgsql. Supabase ofrece PostgreSQL compatible; no se necesitó un paquete NuGet específico de Supabase. La base tenía cuatro tablas de negocio de una versión anterior y un historial de EF sin migraciones registradas. La migración inicial normal no podía ejecutarse a ciegas: habría intentado crear tablas ya existentes.

Objeto	Estado	Diferencia con el modelo actual
Products	1 fila	Price en vez de UnitPrice; faltaban SKU, descripción, categoría, unidad, mínimo, estado y fecha.
Customers	0 filas	Name completo; el modelo separa nombre/apellido. Email y Phone eran obligatorios.
Sales	0 filas	Date en vez de CreatedAtUtc; faltaba estado e información fiscal.
SaleDetails	0 filas	Faltaba LineTotal y confirmar las relaciones requeridas.
ASP.NET Identity	Tablas ausentes	Se necesitaban usuarios, roles, claims, logins y tokens.

2. Conexión y secretos

Se inicializó .NET User Secrets en el proyecto Web y se guardó localmente la cadena compatible con Npgsql. No se guardó la cadena en appsettings.json ni en Git. SSL se configuró con SSL Mode=Require. La conexión se validó leyendo el historial de migraciones antes de cambiar la base.

3. Adaptación aplicada en Supabase

Se añadió la migración 20260929190000_LegacySupabaseBaseline. Reconoce el esquema anterior, verifica su forma, comprueba referencias y datos que no se puedan representar, y se detiene si detecta un caso no contemplado.

Tabla	Cambios y razón
Products	Price pasó a UnitPrice. Se añadieron las columnas actuales. El producto existente se conservó y recibió SKU LEGACY-PROD-<Id> como identificador provisional.
Customers	Name pasó a FirstName; se añadieron LastName, DocumentNumber, DocumentType, Address, Age y CreatedAtUtc. El código divide nombres por el último espacio; sin espacio usaría Pendiente. No existían filas.
Sales	Date pasó a CreatedAtUtc y se añadió Status. Luego la migración fiscal añadió Subtotal, TaxRate, TaxAmount y ExternalReference.
SaleDetails	Se añadió LineTotal; se añadieron referencias e índices con Sales y Products.
Identity	Se crearon siete tablas de ASP.NET Identity con las claves e índices requeridos.
Historial	Se registraron baseline, migración inicial satisfecha y migración fiscal en __EFMigrationsHistory.

Conservación y compatibilidad

No se borraron tablas ni filas. Antes de cambiar el esquema había 1 producto y 0 clientes, ventas o detalles. La migración inicial se marca como satisfecha porque el baseline ya convirtió las tablas y creó Identity; repetir la inicial intentaría crear nuevamente Products, Customers y otras tablas. El baseline se detiene ante esquema parcial o desconocido, Identity preexistente sin historial, emails duplicados, referencias huérfanas o valores fuera del rango soportado.

4. Cómo se ejecutan migraciones

En Program.cs se consulta primero el historial. El baseline solo se ejecuta si todavía no están registradas ni la migración inicial ni el propio baseline; así no se revierte una migración posterior en reinicios. Después Database.MigrateAsync() aplica solo pendientes. En una base nueva, el baseline no-op registra su paso y la migración inicial crea el esquema.

```
dotnet ef migrations add AgregarCampo --project src/Firmeza.Web --startup-project src/Firmeza.Web
```

Los cambios futuros al modelo deben versionarse con EF; no crear las mismas tablas/columnas manualmente en Supabase. La aplicación actualmente migra al arrancar: aceptable para desarrollo y una instancia, pero en producción con varias instancias se recomienda un paso único y controlado del despliegue.

5. Validaciones completadas

Comprobación	Resultado
Conexión Npgsql a Supabase	Correcta; EF pudo consultar el historial.
Migraciones	Baseline, inicial y fiscal aplicadas; EF no muestra pendientes.
Login administrativo	Smoke test autenticado HTTP 200.
Dashboard	Consultas autenticadas a la base HTTP 200.
Build	dotnet build Firmeza.sln --no-restore: 0 advertencias, 0 errores.
Pruebas	dotnet test Firmeza.sln --no-build: 7/7 aprobadas.
Angular y proxy API	Angular HTTP 200; /api/auth/csrf a través del proxy HTTP 200.

6. Archivos nuevos o ajustados

- src/Firmeza.Web/Data/Migrations/20260929190000_LegacySupabaseBaseline.cs — migración de compatibilidad.
- src/Firmeza.Web/Program.cs — orden baseline y migraciones normales.
- src/Firmeza.Web/Firmeza.Web.csproj — identificador local de User Secrets.
- src/Firmeza.Web/appsettings.json — retiro de contraseña administrativa predeterminada débil.
- README.md — conexión, secretos y comportamiento de migración.
- Frontend/package-lock.json — generado para fijar dependencias; no existía lockfile para npm ci.

7. Pendientes antes de producción

- Rotar otra vez la contraseña de Supabase: una contraseña de base se expuso durante el intercambio de configuración. Aunque se rotó durante el trabajo, la nueva credencial también se manipuló en texto visible. Cambiarla en Supabase y actualizar User Secrets/entornos, sin compartirla por chat.
- Cambiar la contraseña administrativa 123456: se solicitó y verificó para entrar; es demasiado débil para producción.
- Configurar secretos de despliegue en un gestor de secretos; User Secrets es solo para desarrollo local.
- Mover migraciones a un paso controlado del pipeline, con respaldo previo; evitar que varias instancias migren simultáneamente.
- Usar permisos mínimos para la cuenta de la aplicación y reservar DDL a migraciones; revisar pooler y políticas/RLS de Supabase.
- Reemplazar valores LEGACY por SKU/documentos reales cuando se definan; verificar recuperación restaurando una copia de seguridad.
- No se desplegó a producción. Configurar HTTPS, dominio, orígenes y observabilidad al publicar.

8. Conclusión

La base está funcional y conectada para el entorno local de pruebas. El esquema actual corresponde al modelo del backend; no quedan migraciones pendientes; el producto previo se conservó; login y consultas del panel funcionaron. La base no necesita otra migración para el modelo actual.

LISTO PARA PRUEBAS LOCALES

Pendiente para producción: rotación de credenciales expuestas, cambiar 123456, configurar secretos del host, probar restauración del respaldo y ejecutar migraciones como paso único de despliegue.

Ejemplo para una futura columna

1. Modificar la entidad/configuración EF. 2. Crear migración con `dotnet ef migrations add`. 3. Revisar SQL y probar en desarrollo/staging con respaldo. 4. Aplicar desde un paso controlado del despliegue. 5. Verificar historial, consultas y pruebas. Evitar cambios manuales paralelos en Supabase.

Informe basado en el estado observado el 1 de octubre de 2026. No contiene contraseñas, cadenas de conexión ni datos personales.